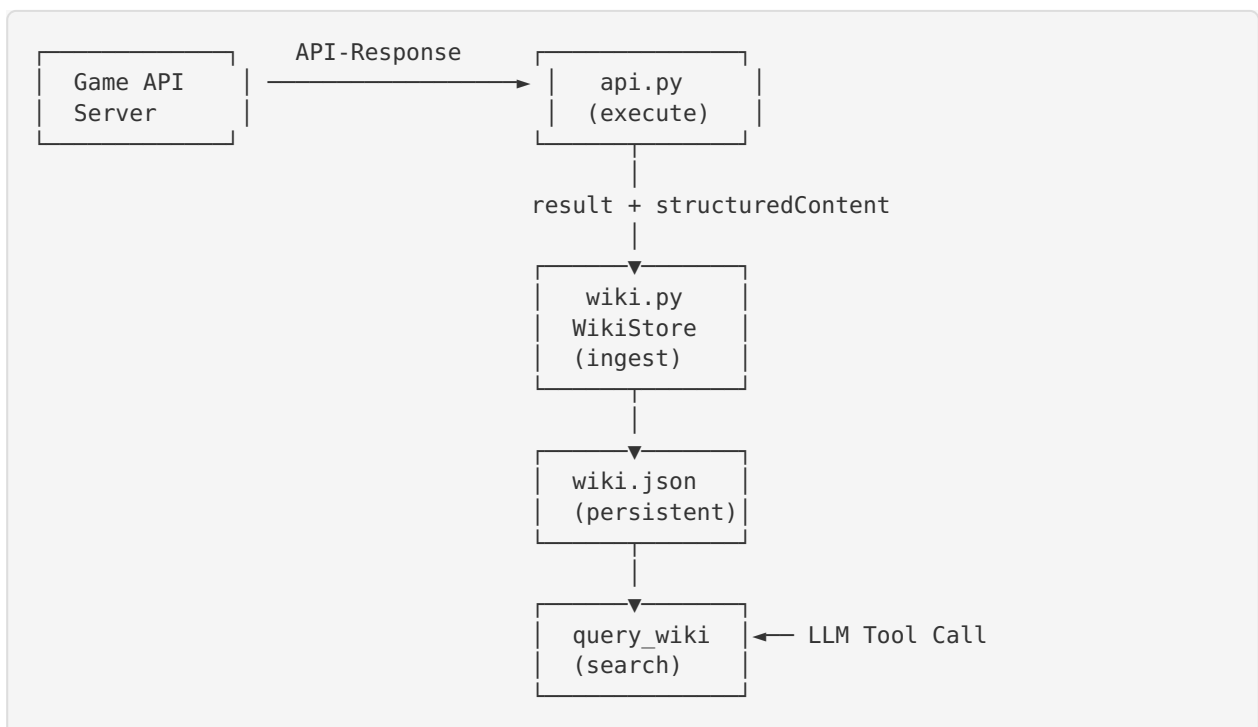


SpaceMolt Wiki Knowledge Base

Übersicht

Das Wiki-System ist eine persistente, selbstlernende Wissensdatenbank für den SpaceMolt Commander Agent. Es extrahiert automatisch Wissen aus jeder Game-Interaktion und ermöglicht dem Agenten, aus seinen Erfahrungen zu lernen, anstatt Informationen immer wieder neu abzufragen.

Architektur



Funktionsweise

1. Automatische Datenextraktion

Jeder API-Call an den Game-Server wird automatisch vom Wiki verarbeitet:

API-Endpoint	Extrahierte Daten
<code>get_system</code>	Systemname, Empire, POIs, Ressourcen
<code>get_map</code>	Alle entdeckten Systeme mit Connections
<code>get_poi</code>	POI-Details, Ressourcen mit Ergiebigkeit
<code>get_base</code>	Stationsinfo, Facilities, Services
<code>mine</code>	Mining-Yield, Ressourcenart, Menge, Erschöpfung
<code>view_market</code>	Preise pro Item/Station
<code>analyze_market</code>	Markt-Insights
<code>craft</code>	Crafting-Ergebnis, XP, Skill-Level
<code>catalog</code>	Items, Rezepte, Module, Schiffe, Facilities
<code>query_intel</code>	Fraktions-System-Intel
<code>query_trade_intel</code>	Fraktions-Handels-Intel
<code>get_skills</code>	Spieler-Skills
<code>get_status</code>	Spielerstatus, Position

2. Datenstruktur

Das Wiki speichert Daten in folgenden Kategorien:

```
{
  "meta": { "version": 1, "total_updates": 42 },
  "systems": { "<system_id>": { "name": "...", "pois": {...}, "empire": "..." } },
  "stations": { "<base_id>": { "name": "...", "system_id": "...", "services":
[...]} },
  "items": { "<item_id>": { "name": "...", "category": "..." } },
  "recipes": { "<recipe_id>": { "name": "...", "inputs": [...], "outputs": [...] } },
  "ship_classes": { "<class_id>": { "name": "...", "build_materials": {...} } },
  "modules": { "<module_id>": { "name": "...", "stats": {...} } },
  "facility_types": { "<type_id>": { "name": "...", "service": "..." } },
  "mining_history": [ { "timestamp": "...", "resource_name": "...", "quantity": 10 } ]
,
  "crafting_history": [ { "timestamp": "...", "recipe": "...", "outputs": [...] } ],
  "market_prices": { "<station_id>": { "<item_id>": [ { "buy_price": 100,
"sell_price": 80 } ] } },
  "intel": { "system_intel": {...}, "trade_intel": {...} },
  "player": { "skills": {...}, "visited_systems": [...] }
}
```

3. Query-Interface

Der LLM kann das Wiki über das `query_wiki` Tool abfragen:

Beispiel-Frage	Suchmechanismus
"systems with asteroid_belt"	POI-Typ-Filter über alle Systeme
"where did I mine Iron Ore"	Mining-History nach Ressource filtern
"best prices for Fuel"	Marktdaten nach Item durchsuchen
"crafting recipes"	Alle bekannten Rezepte auflisten
"stations in system X"	Stationen nach System filtern
"stats"	Wiki-Statistik-Übersicht
"ship classes"	Bekannte Schiffsklassen
"facilities"	Facility-Typen

4. Persistenz

- **Speicherort:** `sessions/{session_name}/wiki.json`
- **Auto-Save:** Nach jeder Datenextraktion
- **Schema-Upgrades:** Neue Felder werden bei Laden automatisch ergänzt
- **Größenbegrenzung:** Mining-History auf 200 Events, Marktdaten auf 20 Snapshots/Item

Nutzung durch den Agenten

Typischer Workflow

1. **Login** → `full_status_check` → Wiki lernt Position, Skills, Status
2. **Catalog Sync** → `catalog_sync` Sequence → Wiki lernt alle Items, Rezepte, Module
3. **Exploration** → `explore_system` → Wiki lernt Systeme, POIs, Ressourcen
4. **Mining** → `mine_and_return` → Wiki lernt Mining-Erträge, Erschöpfung
5. **Trading** → `view_market` → Wiki lernt Preise
6. **Vor jeder Aktion** → `query_wiki` → Agent prüft, was er bereits weiß

Token-Einsparungen

Aktion	Ohne Wiki	Mit Wiki
"Wo gibt es Iron Ore?"	3+ API-Calls + Parsing	1 <code>query_wiki</code> Call
"Welche Rezepte kenne ich?"	<code>catalog(type=recipes)</code>	1 <code>query_wiki</code> Call
"Bester Fuel-Preis?"	Mehrere <code>view_market</code> Calls	1 <code>query_wiki</code> Call
"Mining-Statistik"	Manuelles Tracking	1 <code>query_wiki</code> Call

Konfiguration

Keine separate Konfiguration nötig. Das Wiki wird automatisch beim Start des Commanders initialisiert und mit dem API-Client verbunden.

Erweiterung

Neue Datenquellen können durch Hinzufügen neuer `_ingest_*` Methoden in `WikiStore` erschlossen werden:

```
# In wiki.py
def _ingest_my_new_data(self, data: dict) -> None:
    # Extract and store relevant data
    self._data["my_category"][id] = {...}
    self._mark_dirty()
```

Dann in `_ingest_impl` den neuen Command-Handler registrieren:

```
elif cmd == "spacemolt/my_new_command":
    self._ingest_my_new_data(merged_data)
```